

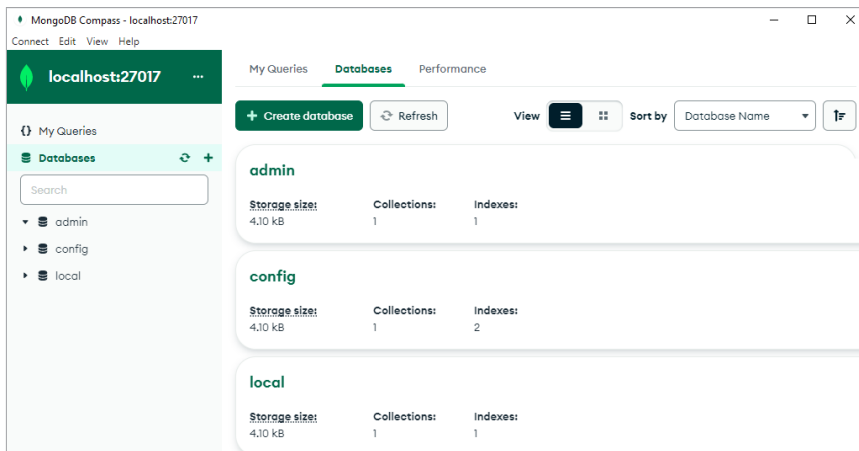


Figure 4: Access the MongoDB services from the local host

Step 3: To work with MongoClient, we need to connect to the default host and port using the following command:

```
$ client = pymongo.  
MongoClient("mongodb://  
localhost:27017/")
```

Step 4: MongoDB allows multiple databases to coexist on a single server. Databases in PyMongo are accessed via MongoClient instances with dictionary-style access:

```
$ db = client["mydatabase"]
```

Table 2: List of MongoDB commands used in IoT

Command	Description
<code>db.createCollection("sensors")</code>	Creates a collection of 'sensors' to store data
<code>db.insertOne</code>	Adds a document to a collection
<code>db.insertMany</code>	Adds multiple files to a collection
<code>db.sensors.find({ sensorId: s })</code>	Retrieves sensor readings where the sensor ID is 's'
<code>db.aggregate</code>	Performs aggregation operations on the data
<code>db.updateOne</code>	Updates a filter-matched document
<code>db.deleteOne</code>	Deletes a filter-matched document
<code>db.deleteMany</code>	Deletes multiple filter-matched documents

for IoT applications. Table 2 lists some popular MongoDB commands commonly used in IoT scenarios.

Programming MongoDB using Python

To work with MongoDB in Python, you can use the PyMongo library, the official MongoDB driver for Python. PyMongo provides a simple and intuitive API for interacting with MongoDB from your Python code.

Step 1: Install PyMongo. PyMongo can be set up with the help of *pip*, Python's package manager. This can be done using the following command:

```
$ pip install pymongo
```

Step 2: The PyMongo package can now be imported by running the following command:

```
$ import pymongo
```

Step 5: Now, access a collection within the database:

```
$ collection = db["yourcollection"]
```

A collection in MongoDB is a set of related documents that functions similar to a table in a traditional database. Python's method for accessing collections is identical to that used for databases.

Step 6: In this step, you can insert a document into a collection. Data in MongoDB is represented (and stored) using JSON-style documents. In PyMongo, we use dictionaries to represent documents. As an example, the following dictionary inserts a document in a MongoDB collection:

```
$document = {"Name": "OSFY", "Month": 07}  
$ collection.insert_one(document)
```

MongoDB is a NoSQL database widely used because of its document-oriented design. Built with horizontal scalability across multiple servers, it can easily handle massive amounts of data. Industries using the Internet of Things often need to process and analyse streaming data in real-time. MongoDB's real-time data processing, querying, and aggregation capabilities facilitate prompt analysis and decision-making. 

References

- <https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-windows/>
- <https://www.mongodb.com/docs/manual/reference/>

By: Dr Aditya Bhardwaj

The author is a B. Tech, M. Tech, and PhD in CSE. He works as assistant professor in the School of Computer Science Engineering and Technology at Bennett University, Greater Noida. He is experienced in cloud computing and open source technology.